

Alright — I'll turn this into a **complete end-to-end system blueprint + starter implementation** so you can actually build it instead of just planning it.

This is your **MLB Strikeout Edge App (MVP → Model → Product)**.

1. FULL SYSTEM ARCHITECTURE

Overview

You are building 4 layers:

1) Data Layer

- MLB stats + odds data ingestion

2) Model Layer

- Expected strikeouts (λ)
- Probability engine (Poisson / ML hybrid)

3) API Layer

- FastAPI backend serving predictions

4) App Layer

- Simple dashboard showing “bet / no bet”
-

2. PROJECT STRUCTURE (REALISTIC MVP)

```
mlb-edge-app/  
|  
├── backend/  
|   └── main.py
```

```
|
| ├── model.py
| ├── data.py
| ├── odds.py
| └── utils.py
|
| ├── frontend/
| │   └── app.js (React / Next.js)
|
| ├── data/
|
| ├── requirements.txt
| └── README.md
```

3. DATA PIPELINE (what feeds your model)

MLB DATA SOURCES

You combine:

- MLB Stats API (pitching, games)
 - FanGraphs (K%, SwStr%)
 - Baseball Savant (pitch data)
 - Odds API (sportsbook lines)
-

DATA FIELDS YOU STORE

Pitcher table

- name
- K/9
- K%
- SwStr%
- avg innings/start
- pitch count avg
- last 3 starts innings

Opponent table

- team K% vs RHP/LHP
- OBP
- wRC+

Game context

- park factor
 - weather
 - implied runs (Vegas)
-

4. CORE MODEL (HYBRID SYSTEM)

You should NOT rely on only one method.

You use:

◆ STEP 1 — Expected Strikeouts

$$\lambda = (K/9) \times IP \times M_{\text{opp}} \times M_{\text{park}} \times M_{\text{form}}$$

Where:

- M_{opp} = opponent strikeout factor
 - M_{park} = park adjustment
 - M_{form} = recent form multiplier
-

◆ STEP 2 — Probability Engine

Start simple (Poisson):

$$P(X=k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Then compute:

- $P(\text{Over line})$

- $P(\text{Under line})$
-

◆ STEP 3 — FAIR ODDS

$$\text{Fair Odds} = \frac{1}{P}$$

Convert to American odds for comparison.

◆ STEP 4 — EDGE CALCULATION

$$\text{Edge} = \text{Implied Probability} - \text{Model Probability}$$

Only bet if:

- $\text{Edge} \geq 3\text{--}5\%$
-



5. BACKEND CODE (FASTAPI MVP)

main.py

```
from fastapi import FastAPI
from model import predict_strikeouts

app = FastAPI()

@app.get("/predict")
def predict(pitcher: str, opponent: str, line: float):
    result = predict_strikeouts(pitcher, opponent, line)
    return result
```

model.py

```

import math

def poisson_pmf(k, lam):
    return (lam**k * math.exp(-lam)) / math.factorial(k)

def over_probability(lam, line):
    threshold = int(line) + 1
    return sum(poisson_pmf(k, lam) for k in range(threshold, 15))

def predict_strikeouts(pitcher, opponent, line):

    # --- simplified demo values (replace with real data pull) ---
    k_per_9 = 9.8
    innings = 5.8
    opp_factor = 0.95

    lam = (k_per_9 / 9) * innings * opp_factor

    p_over = over_probability(lam, line)
    fair_odds = 1 / p_over if p_over > 0 else 0

    return {
        "pitcher": pitcher,
        "opponent": opponent,
        "line": line,
        "expected_strikeouts": lam,
        "over_probability": p_over,
        "fair_odds": fair_odds
    }

```



6. DATA FETCH MODULE (placeholder)

data.py

```

def get_pitcher_stats(name):
    return {
        "k_per_9": 9.5,
        "innings": 5.7,
        "swstr": 12.5
    }

```

```
}
```

```
def get_opponent_factor(team):  
    return 0.92
```

7. FRONTEND (VERY SIMPLE REACT)

```
import { useState } from "react";  
  
export default function App() {  
    const [result, setResult] = useState(null);  
  
    const fetchPrediction = async () => {  
        const res = await fetch(  
            "/predict?pitcher=Cease&opponent=NY Yankees&line=7.5"  
        );  
        const data = await res.json();  
        setResult(data);  
    };  
  
    return (  
        <div>  
            <button onClick={fetchPrediction}>  
                Predict Strikeouts  
            </button>  
  
            {result && (  
                <pre>{JSON.stringify(result, null, 2)}</pre>  
            )}  
        </div>  
    );  
}
```

8. HOW TO MAKE THIS “SHARP” (REAL EDGE SYSTEM)

Once MVP works, upgrade in this order:

LEVEL 2 — REAL EDGE FEATURES

1. Pitch count prediction model

Instead of fixed innings:

- predict pitch count → innings ceiling

2. SwStr% weighting

Swinging strike rate improves K prediction accuracy massively.

3. Lineup confirmation engine

Detect:

- high-K hitters added → boost pitcher Ks
 - low-K hitters → downgrade Ks
-

LEVEL 3 — PROFESSIONAL EDGE

1. Monte Carlo simulation (upgrade Poisson)

- simulate full game 10,000 times

2. Live betting model

- inning-by-inning K probability updates

3. Market comparison engine

Pull:

- DraftKings
- FanDuel
- bet365

Find best line automatically.



9. WHAT ACTUALLY MAKES THIS MONEY

Not prediction accuracy.

Money comes from:

- mispriced innings projections
- late lineup changes
- weather/umpire inefficiencies
- slow sportsbook adjustments



FINAL REALITY

If you build this correctly:

- You are not “betting”
- You are **pricing probabilities better than sportsbooks**

That’s the entire game.



IF YOU WANT NEXT STEP

I can now help you build:

👉 Option A

Full GitHub-ready project (copy/paste runnable)

👉 Option B

Upgrade to ML model (XGBoost > Poisson)

👉 **Option C**

Add live betting + automation bot

👉 **Option D**

Turn this into a SaaS (sell it)

Just tell me which direction you want.